#E Include the context string and the current block counter
#F Copy the new HMAC tag to the next block of output

You can now use this to generate as many keys as you want from an initial HMAC key. For example, you can open the Main.java file and replace the code that loads the AES encryption key from the keystore with the following code that derives it from the HMAC key instead:

```
var macKey = keystore.getKey("hmac-key",
"changeit".toCharArray());
var encKey = HKDF.expand(macKey, "token-encryption-key",
        32, "AES");
```

> **WARNING** A cryptographic key should be used for a single purpose. If you use a HMAC key for key derivation you should not use it to also sign messages. You can use HKDF to derive a second HMAC key to use for signing.

You can generate almost any kind of symmetric key using this method, making sure to use a distinct context string for each different key. Key pairs for public key cryptography generally can’t be generated in this way, because the keys are required to have some mathematical structure that is not present in a derived random key. However, the Salty Coffee library used in chapter 6 contains methods for generating key pairs for public key encryption and for digital signatures from a 32-byte seed, which can be used as follows: